

AI-BASED VIRTUAL CAMERA TRACKING SYSTEM

User Manual

Installation, Operation & Troubleshooting Guide

Version 1.0 — September 2026

Table of Contents

1. Overview
2. Installation
3. Running the Application
4. GUI Walkthrough
5. Configuring Parameters
6. Example Scenarios
7. Troubleshooting

1. Overview

This application is a virtual testbed for the coarse-alignment stage of a Free-Space Optical Communication (FSOC) pointing system. It simulates a moving optical beacon inside a 2000x2000 pixel virtual world, drives a simulated pan-tilt camera to track it, and lets you inject realistic disturbances (image noise, atmospheric conditions, camera jitter) to evaluate tracking robustness. It also supports replaying recorded video files through the same tracking pipeline for benchmark scoring against ground truth.

The system has two parts: a Python/FastAPI backend that runs the simulation and tracking algorithms, and a browser-based frontend (React) that displays the live view and exposes all controls.

2. Installation

The backend requires Python 3.10+ and the frontend requires Node.js 18+. Steps are the same shape on all platforms; only the shell commands differ.

2.1 Prerequisites

Requirement	Windows	macOS	Linux
Python	3.10+ from python.org (check 'Add to PATH')	3.10+ via python.org or brew install python	3.10+ via package manager (apt/dnf)
Node.js	18+ from nodejs.org	18+ via nodejs.org or brew install node	18+ via nodesource or nvm
Git	git-scm.com installer	brew install git or Xcode CLT	apt/dnf install git

2.2 Clone the Repository

```
git clone https://github.com/ayushhroy77/SIH26169.git
cd SIH26169
```

2.3 Backend Setup

From the repository root:

```
# Windows (PowerShell)
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt

# macOS / Linux
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

2.4 Frontend Setup

From the repository root, in a separate terminal:

```
cd frontend # or the root folder, if that is where package.json lives
npm install
```

Note: this repository currently contains a stray Next.js scaffold under frontend/ alongside the actual Vite + React project at the repository root. Confirm which folder holds the up-to-date package.json before running npm install, since installing into the wrong folder will not start the real application.

2.5 Verify the Install

- Backend: run `python -m uvicorn main:app --reload` from inside backend/ and confirm it starts without errors on `http://localhost:8000`.
- Frontend: run `npm run dev` and confirm it starts on `http://localhost:5173` (Vite's default) without build errors.

3. Running the Application

3.1 Start the Backend

```
cd backend
source venv/bin/activate    # Windows: venv\Scripts\activate
python -m uvicorn main:app --reload --port 8000
```

3.2 Start the Frontend

```
cd frontend # or repository root, per 2.4
npm run dev
```

3.3 Open the App

Navigate to the URL printed by the frontend dev server (typically <http://localhost:5173>). The app connects to the backend automatically over WebSocket; if it does not connect, see Troubleshooting section 7.2 (proxy configuration).

4. GUI Walkthrough

The layout follows a standard viewport-left, controls-right pattern.

4.1 Scene Canvas (left)

- Live 640x480 camera view, refreshed at 30 Hz
- Green crosshair marks the tracked beacon; its size scales with configured target size
- Magenta reticle marks the fixed boresight (camera center) plus a dynamic error ring showing current tracking error
- Semi-transparent rectangle shows the active camera viewport against the full 2000x2000 world
- Pan/tilt angle readout, updated live (e.g. 'Pan: 0.5 deg / Tilt: -0.3 deg')

4.2 Control Panel (right)

- Target configuration: size slider (5-20 px), shape (square/circle/crosshair), initial location mode
- Motion selector: straight line, circular, figure-8, random walk, spiral, sinusoidal
- Disturbance controls: noise type/intensity sliders (salt-and-pepper, Gaussian, Poisson), atmospheric dropdown (clear/haze/fog/rain/low-light) with a 0-100% severity slider, camera jitter slider
- Servo tuning: Kp/Ki/Kd sliders for the pan-tilt controller
- Scenario presets: Clean, Fog Stress, or Custom

4.3 Metrics Dashboard

- FPS counter (top-right), blinks orange if it drops below 20 FPS
- Lock-state indicator: Searching (yellow) / Acquiring (blue) / Tracking (green) / Lost (red)
- Live centroiding error readout and a bar-graph gauge (turns red above 10 px)
- Lock status text, e.g. 'Locked (99.2%)' or 'Lost'
- Export Report button — downloads the session's performance log as JSON

4.4 Keyboard Shortcuts

Key	Action
Space	Play / Pause simulation
R	Reset to default parameters
E	Export performance report

5. Configuring Parameters

5.1 Target Parameters

Parameter	Range	Default	Effect
Target size	5-20 px	10x10 px	Larger targets are easier to detect but less representative of a distant beacon
Target shape	Square / Circle / Crosshair	Square	Cosmetic; detection uses intensity + shape, not exact silhouette
Initial location	Random / Center / Click-to-place	Random	Controls where the beacon starts each run
Motion type	6 trajectories	Straight line	Determines how the beacon moves; harder trajectories (fig-8, random walk) stress the servo more

5.2 Disturbance Parameters

Parameter	Range	What it simulates
Salt & pepper density	0-20%	Dead/hot pixel defects
Gaussian noise std dev	0-20 px	Sensor read noise
Poisson scale	1-100	Photon shot noise
Atmospheric condition	Clear / Haze / Fog / Rain / Low light	Beam attenuation and contrast loss
Atmospheric severity	0-100%	How strong the selected condition is
Camera jitter	0-20 px	Platform vibration

5.3 Servo Gains

Kp, Ki and Kd sliders adjust the pan-tilt PID controller directly. The shipped defaults (Kp=0.12, Ki=0.005, Kd=0.02) are tuned for critically-damped response under 5 deg/s max speed; raising Kp increases responsiveness at the cost of overshoot, raising Kd damps oscillation but can slow response, and Ki should be left small since large values risk integrator windup even with anti-windup clamping in place.

6. Example Scenarios

6.1 Scenario: Clean Tracking

Setup: motion = straight line, all disturbances at 0%.

Expected result: acquisition in well under 1 second, tracking error staying under ~5 px, lock retention above 99%.

6.2 Scenario: Fog Stress

Setup: motion = random walk, atmospheric = Fog at 70-100% severity, Gaussian noise at moderate level.

Expected result: detection rate drops modestly (measured around 94% in severe fog versus 99.5% clear) but the system should still re-acquire lock within about a second after any loss, rather than staying lost.

6.3 Scenario: Video Ingestion Benchmark

1. Switch to Benchmark / Video Ingestion mode in the app.
2. Upload an MP4 or AVI file (e.g. beacon_test.mp4).
3. Confirm the frame stream begins and the tracker's crosshair follows the beacon in the uploaded footage.
4. Watch the metrics panel populate RMSE, lock retention, and FPS as the video plays through.
5. At end of video, click Export Report to download the session's JSON/CSV performance log.

7. Troubleshooting

7.1 Frontend loads but shows no video / "Disconnected"

- Confirm the backend is running and printed 'Uvicorn running on http://0.0.0.0:8000' with no errors.
- Confirm the frontend dev server's proxy is configured to forward /api and /ws requests to the backend (see 7.2) — without this, the browser cannot reach the backend even though both processes are running.

7.2 WebSocket fails to connect

If the frontend build tool is Vite, its dev server needs an explicit proxy entry so that browser requests to /api and /ws are forwarded to the backend on port 8000, instead of being handled (and rejected) by the frontend server itself. Add this to vite.config.ts:

```
server: {
  proxy: {
    '/api': { target: 'http://localhost:8000', changeOrigin: true },
    '/ws': { target: 'ws://localhost:8000', ws: true },
  },
},
```

After adding this, restart the frontend dev server and confirm the WebSocket connects to ws://localhost:8000/ws/video/<id> (or /ws/telemetry for live mode) in the browser's network tab.

7.3 FPS counter blinking orange / low frame rate

- Close other GPU/CPU-heavy applications; the simulation and detection pipeline are CPU-bound.
- Reduce disturbance severity — heavier noise/atmosphere processing costs more time per frame.
- Confirm you are not running the backend in a debug/reload loop that is also serving other requests.

7.4 Target lock keeps dropping

- Check whether disturbance severity is very high — some loss is expected under severe fog or heavy noise.
- Verify the servo gain sliders have not been set to extreme values (very high Kp can cause oscillation that looks like repeated loss).
- Try the Reset (R) shortcut to return to default parameters and confirm the issue persists.

7.5 Video upload fails or does not play

- Confirm the file is MP4 or AVI — other containers are not currently supported.
- Check the backend console for an error opening the file (a corrupted or zero-frame video will fail at VideoSource.reset()).
- Try a shorter test clip first to rule out a large-file timeout.

7.6 Two frontend folders in the repository

The repository currently contains both a root-level Vite + React project (the real, running application) and a frontend/ folder with an unused Next.js scaffold. If npm install or npm run dev behaves unexpectedly, confirm which folder you are in — only the root-level project is the maintained application as of this manual's writing.